

CT-157

Data Structure and Algorithms Design

Arrays

Dr. Raheela Asif

Department of Software Engineering, NED University of Engineering & Technology, Karachi

DATA & STRUCTURES

DATA:

“Data is a collection of facts, numbers, letters or symbols that the computer process into meaningful information.”

STRUCTURE:

“The arrangement of and relations between the parts or elements.”

DATA STRUCTURES

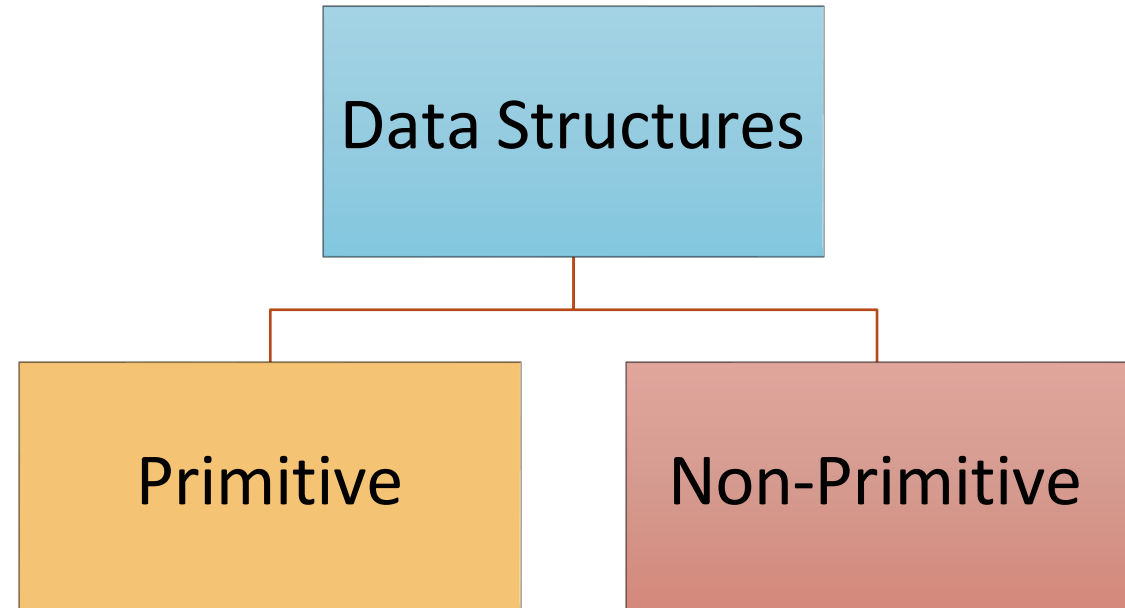
“Data structure is representation of the logical relationship existing between individual elements of data.”

OR

“Data structure is a specialized format for organizing and storing data in memory that considers not only the elements stored but also their relationship to each other”.

CLASSIFICATION OF DATA STRUCTURES

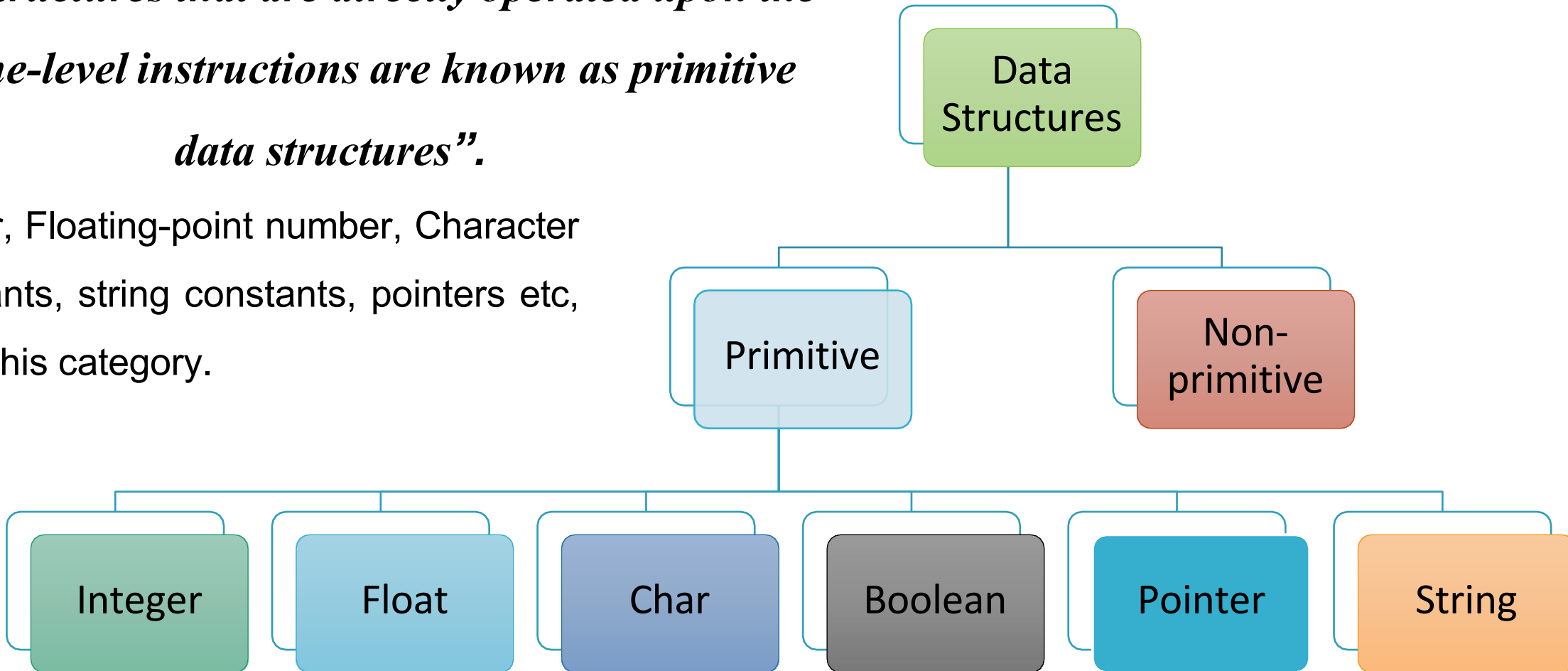
- Data structure are normally divided into two broad categories:
 1. Primitive Data Structure
 2. Non-Primitive Data Structure



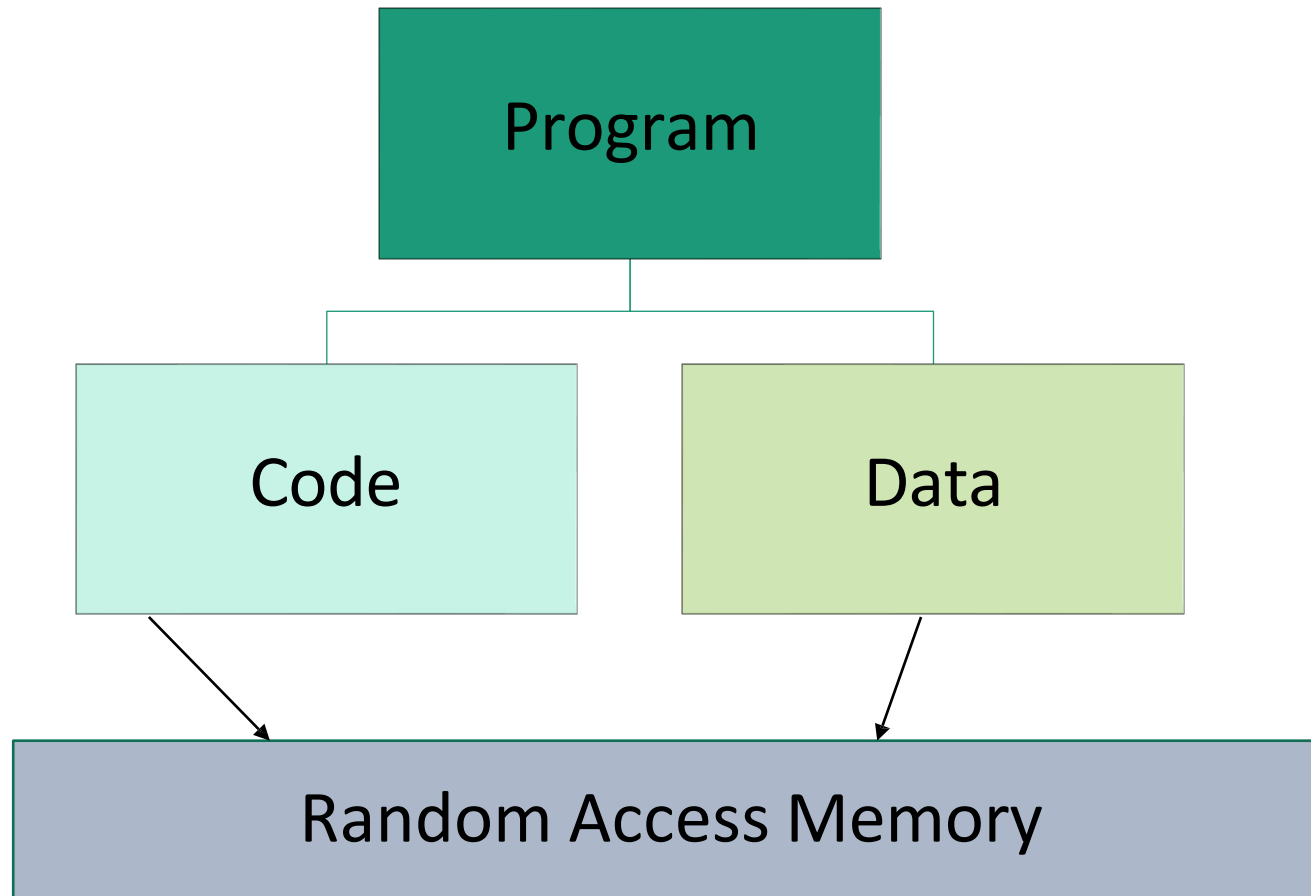
PRIMITIVE DATA STRUCTURES

“Data structures that are directly operated upon the machine-level instructions are known as primitive data structures”.

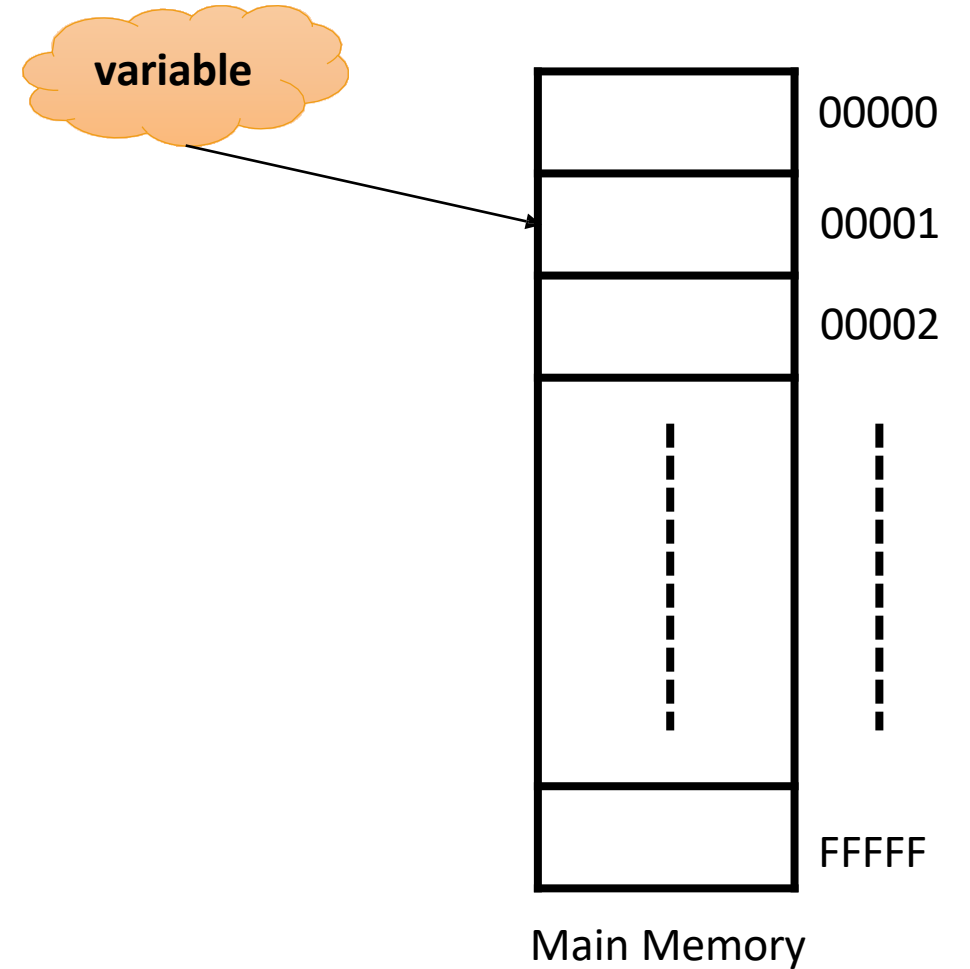
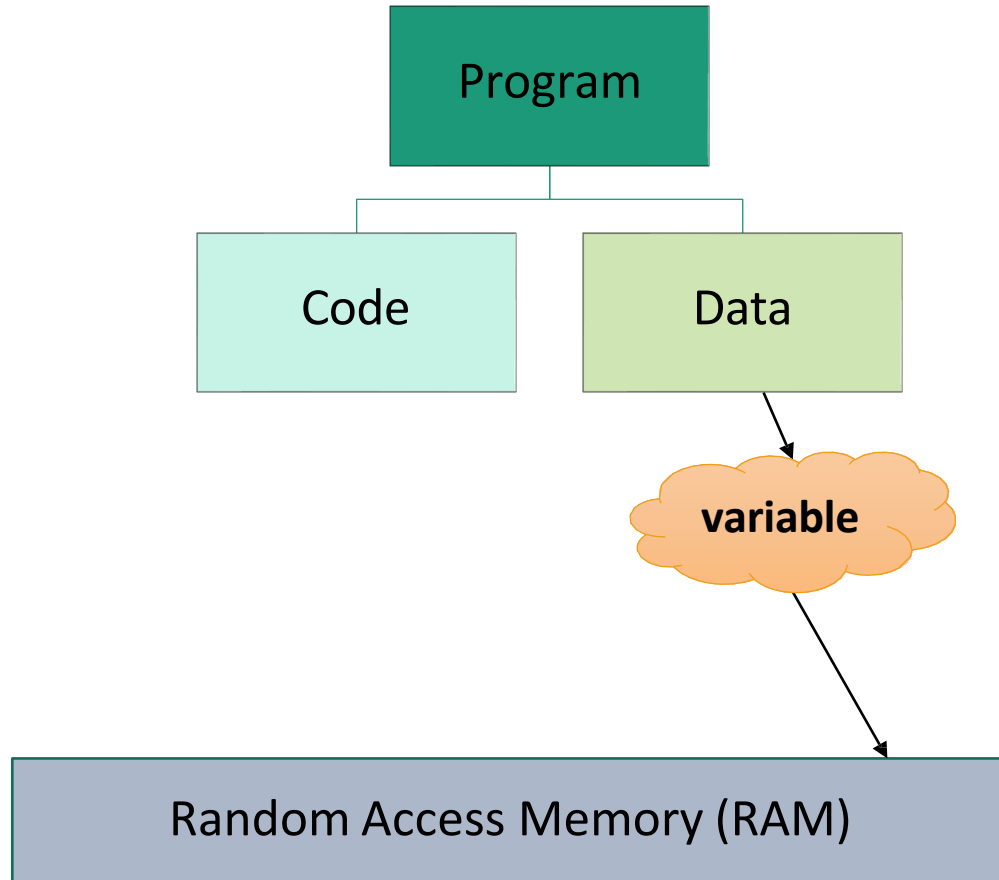
- Integer, Floating-point number, Character constants, string constants, pointers etc, fall in this category.



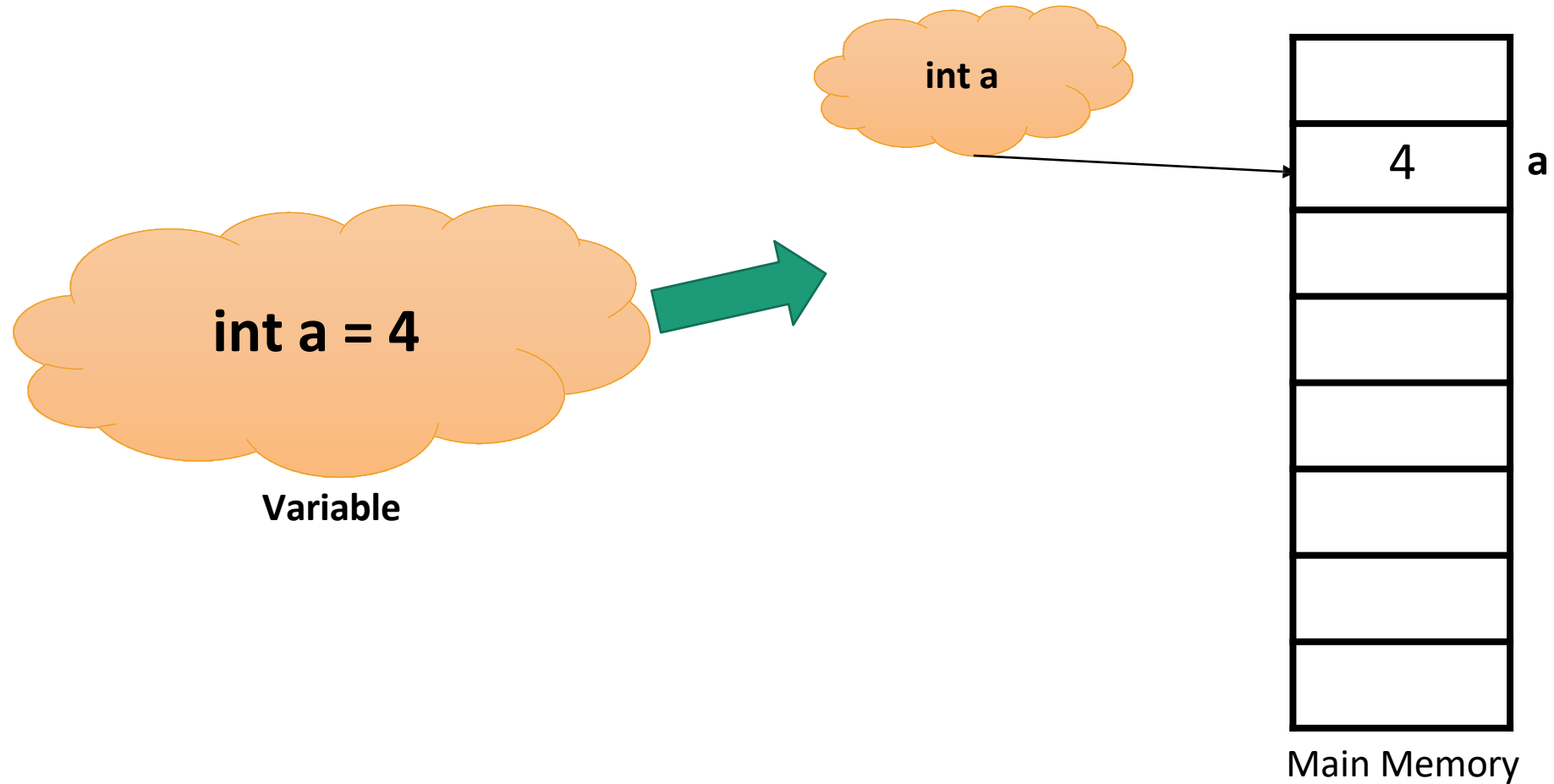
DATA STORAGE



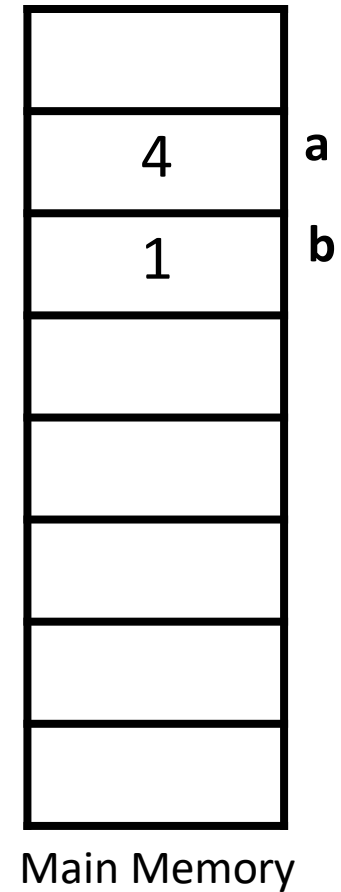
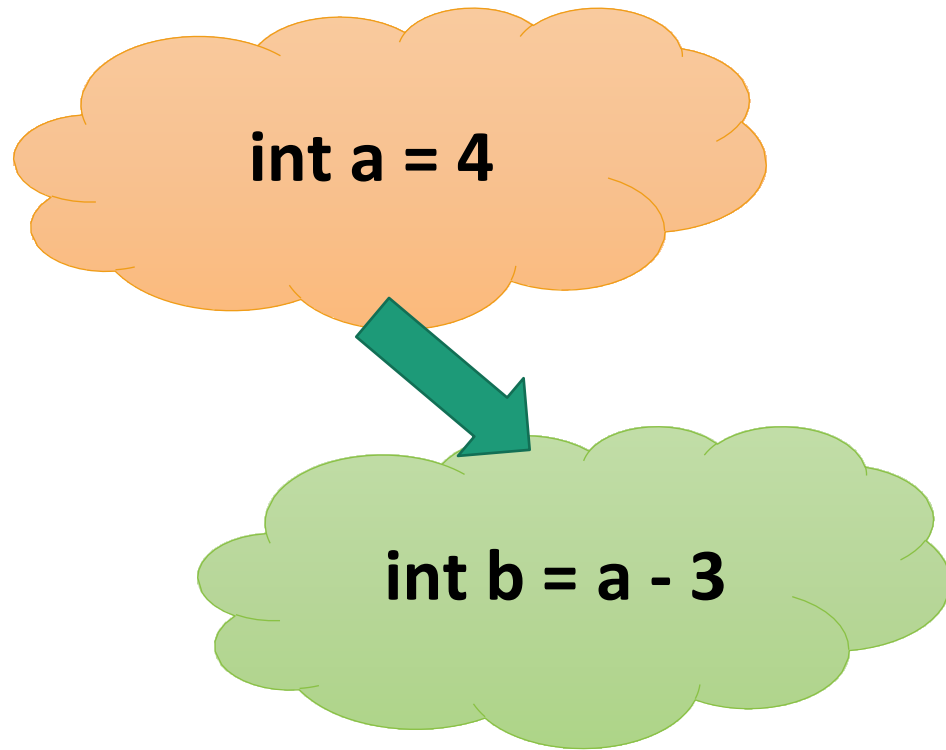
DATA STORAGE



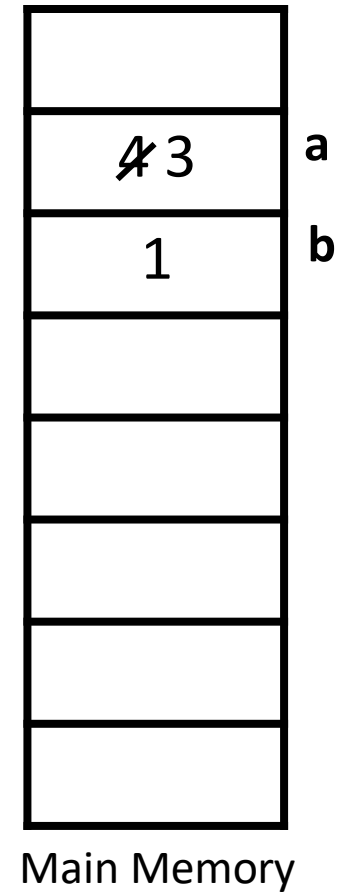
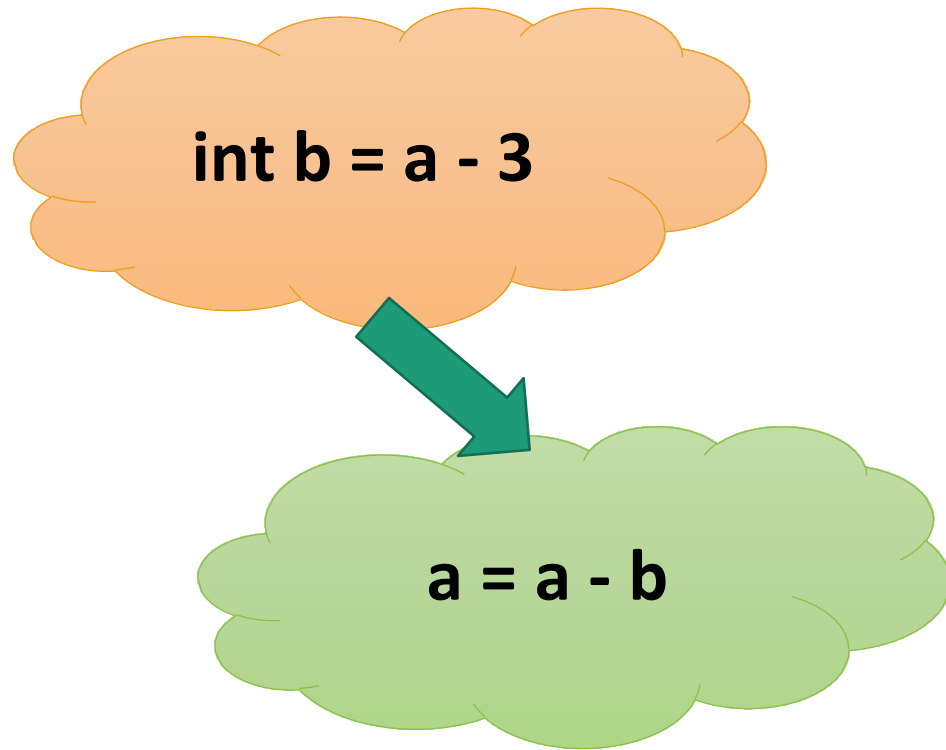
DATA STORAGE



DATA STORAGE



DATA STORAGE



NON-PRIMITIVE DATA STRUCTURES

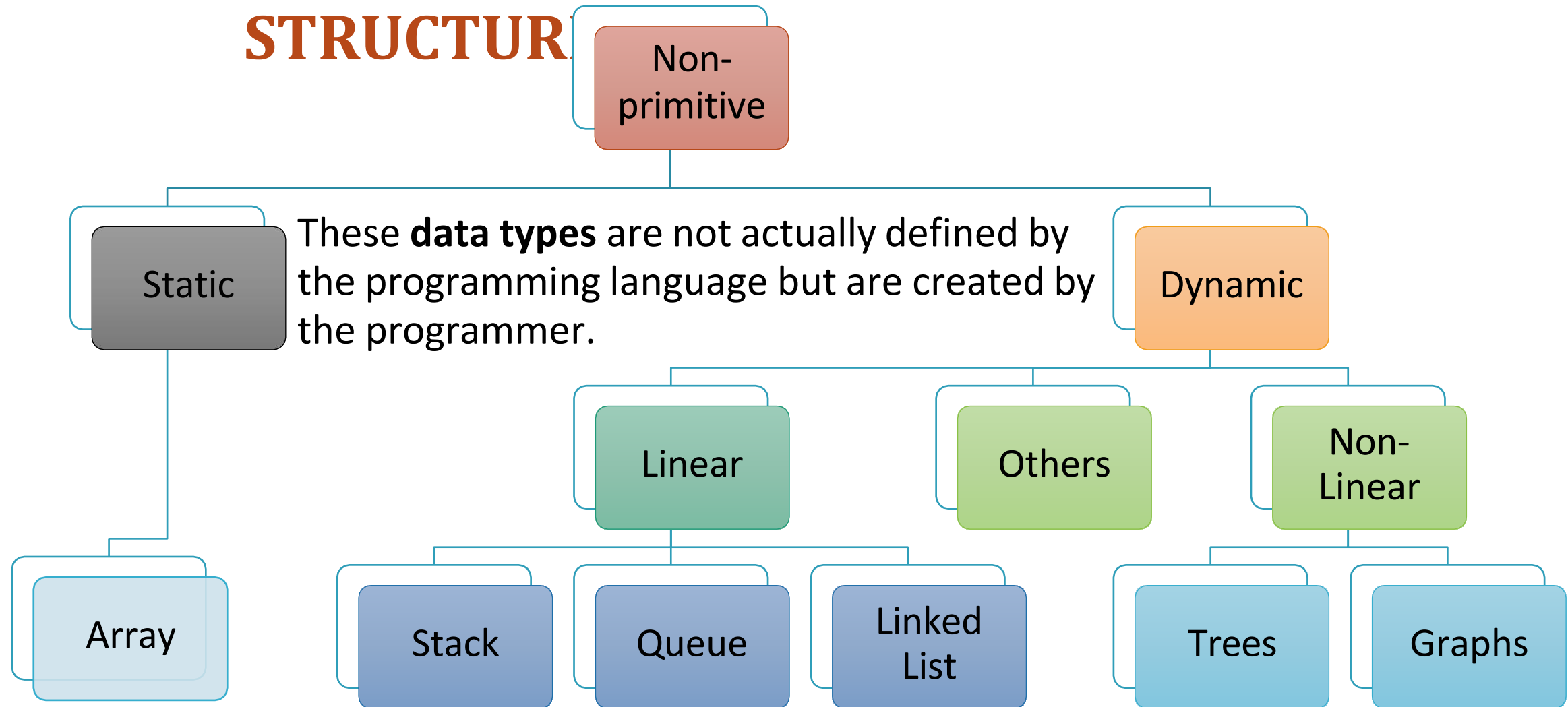


- There are more sophisticated data structures.

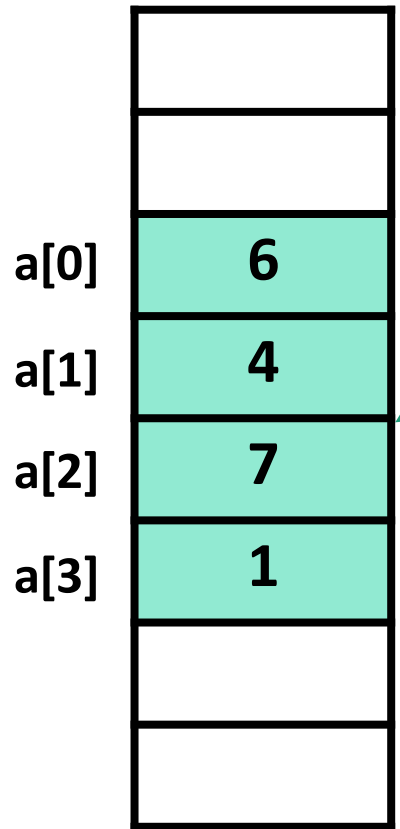
“ The Data structures that are derived from the primitive data structures are called Non-primitive data structure.”

- The non-primitive data structures emphasize on structuring of a group of homogeneous (same type) or heterogeneous (different type) data items.

NON-PRIMITIVE DATA STRUCTURE



ARRAY



Main Memory

int a[4]

- An array is defined as a set of finite number of homogeneous elements or same data items.
- It means an array can contain one type of data only, either all integer, all float-point number or all character.
- The elements of linear array are stored in consecutive memory locations. It is shown below:

int a[4] =
{6,4,7,1}

array

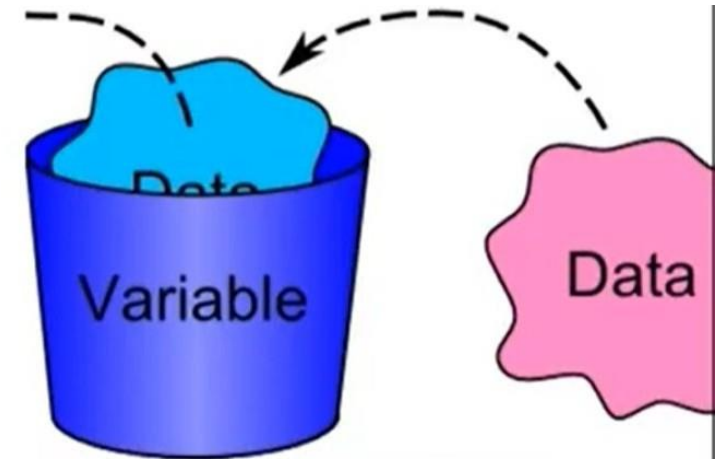
ARRAY

- The operations that can be performed on an array are:
 1. Create
 2. Insert values
 3. Delete values
 4. Insert at a certain index
 5. Traverse

NODE

- The data structure we use to hold user defined data types is called Node.
- The operations and values of user defined data types are it specified in the language itself but is specified by the user.
- Like:
 - Structure

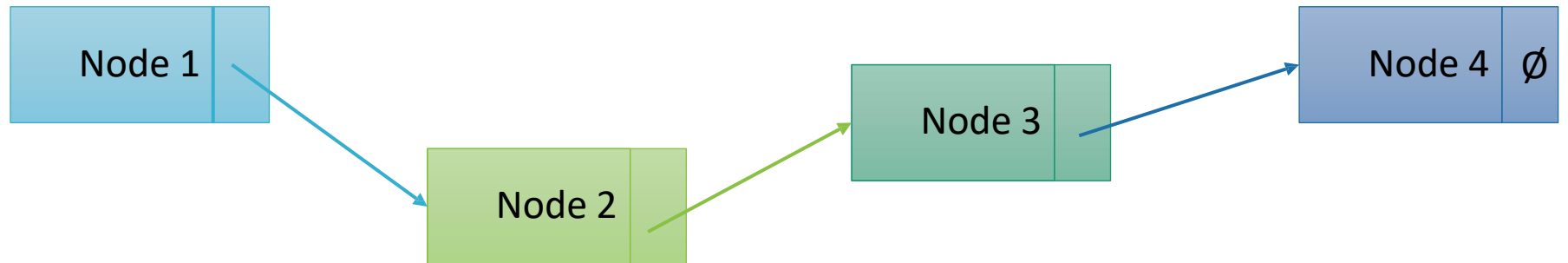
```
struct point {  
    int x;  
    int y;  
};
```



For example, by using structures, we are defining our own type by combining data types.

NODE

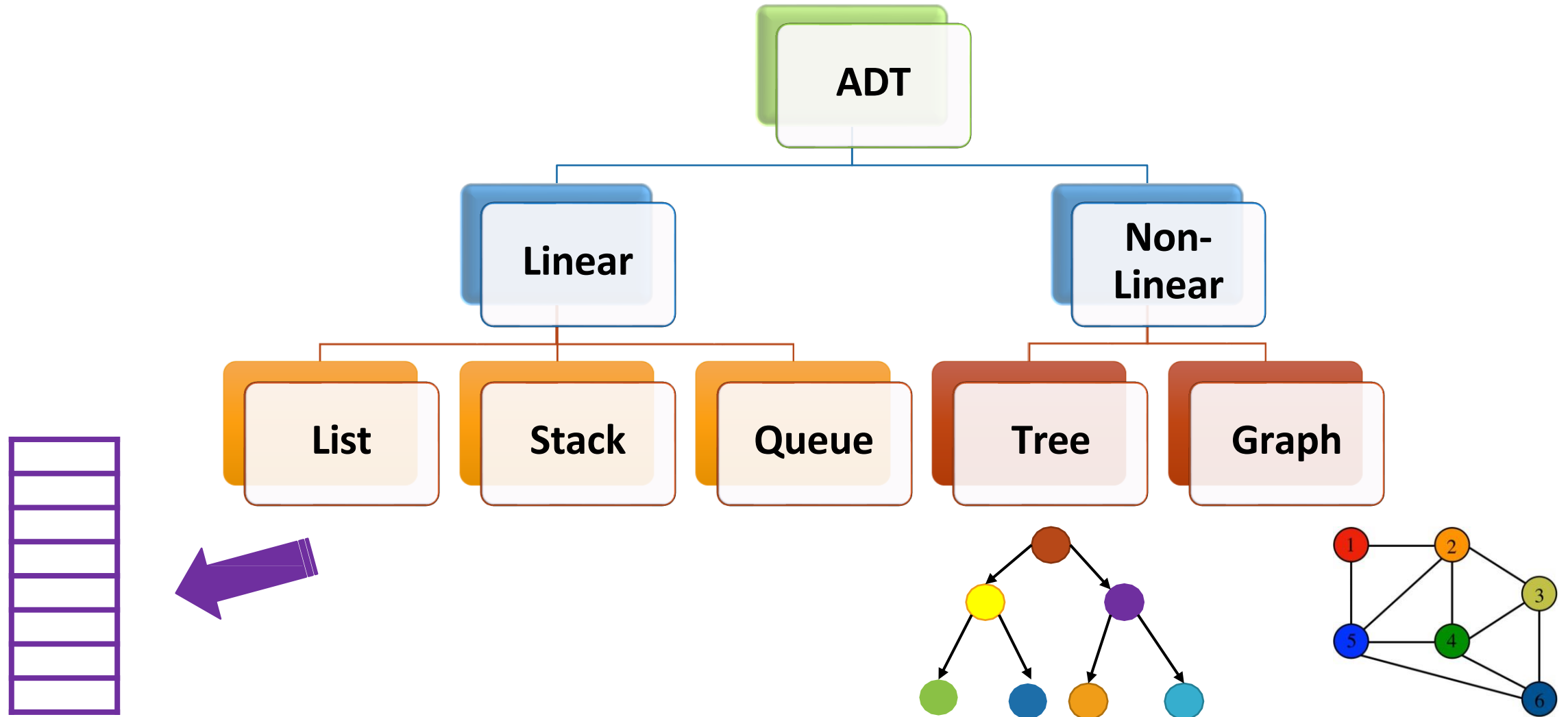
- The operations that can be performed on an array are:
 1. Create
 2. Insert values
 3. Delete values
 4. Traverse



ABSTRACT DATA TYPES

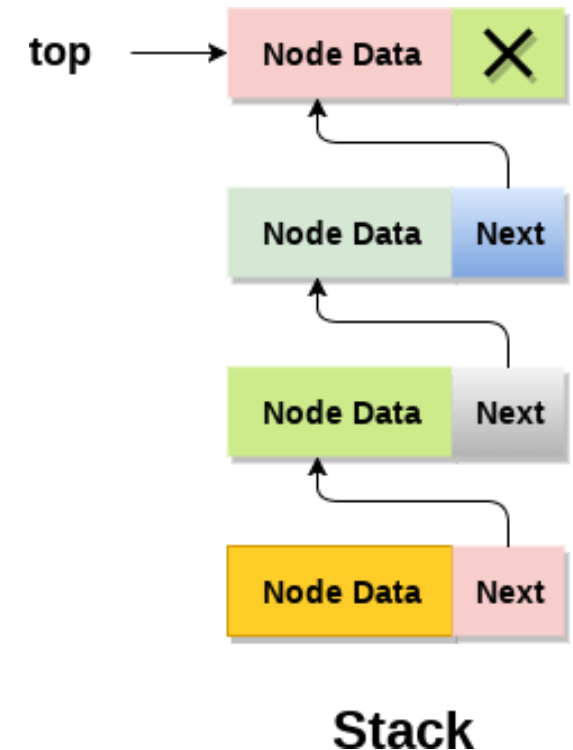
- A data structure is for organizing your data.
- An abstraction:
 - ✓ does not worry about implementation details
 - ✓ simplicity
 - ✓ intuitive understandability
 - ✓ user friendly

ABSTRACT DATA TYPES



ABSTRACT DATA TYPES (ADTs)

- ADTs are just skeletons.
- There are multiple ways to implement an ADT.
- A Stack ADT can be implemented using:
 - Static Arrays
 - Dynamic Arrays
 - Linked Lists



BASIC OPERATIONS ON DATA STRUCTURES

- Each of these operation has separate algorithms for each data structure.
 - ✓ Read/traverse values
 - ✓ Search values
 - ✓ Insert values
 - ✓ Delete values
 - ✓ Sort values
 - ✓ Swap two values
 - ✓ Update values
 - ✓ Etc.

How Memory Works...???

- You can imagine of memory as a chest of drawers to store your things.
- Each drawer can hold one element.
- You want to store two things, so you ask for two drawers.



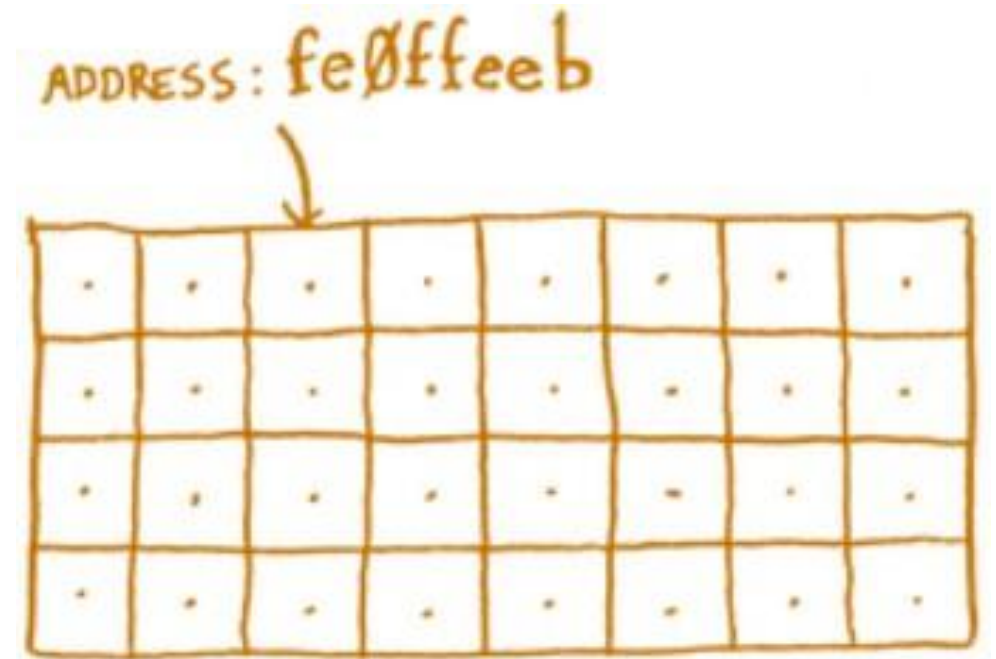
How Memory Works...???

- You store your two things here.



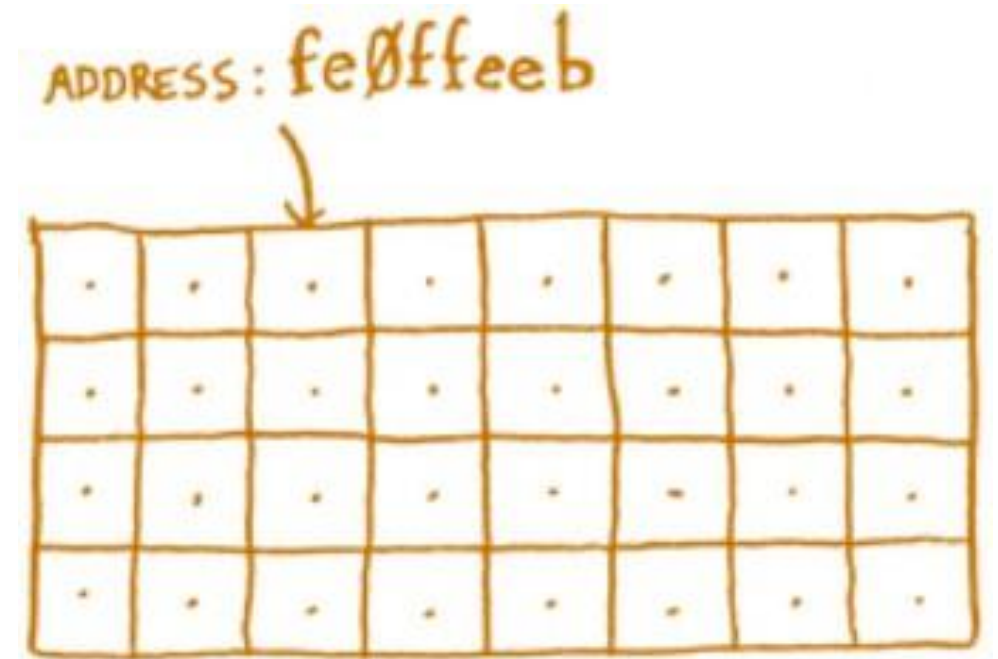
How Memory Works...???

- This is basically how your computer's memory works.
- Your computer looks like a giant set of drawers, and each drawer has an address.
- Each address is some hexadecimal number.



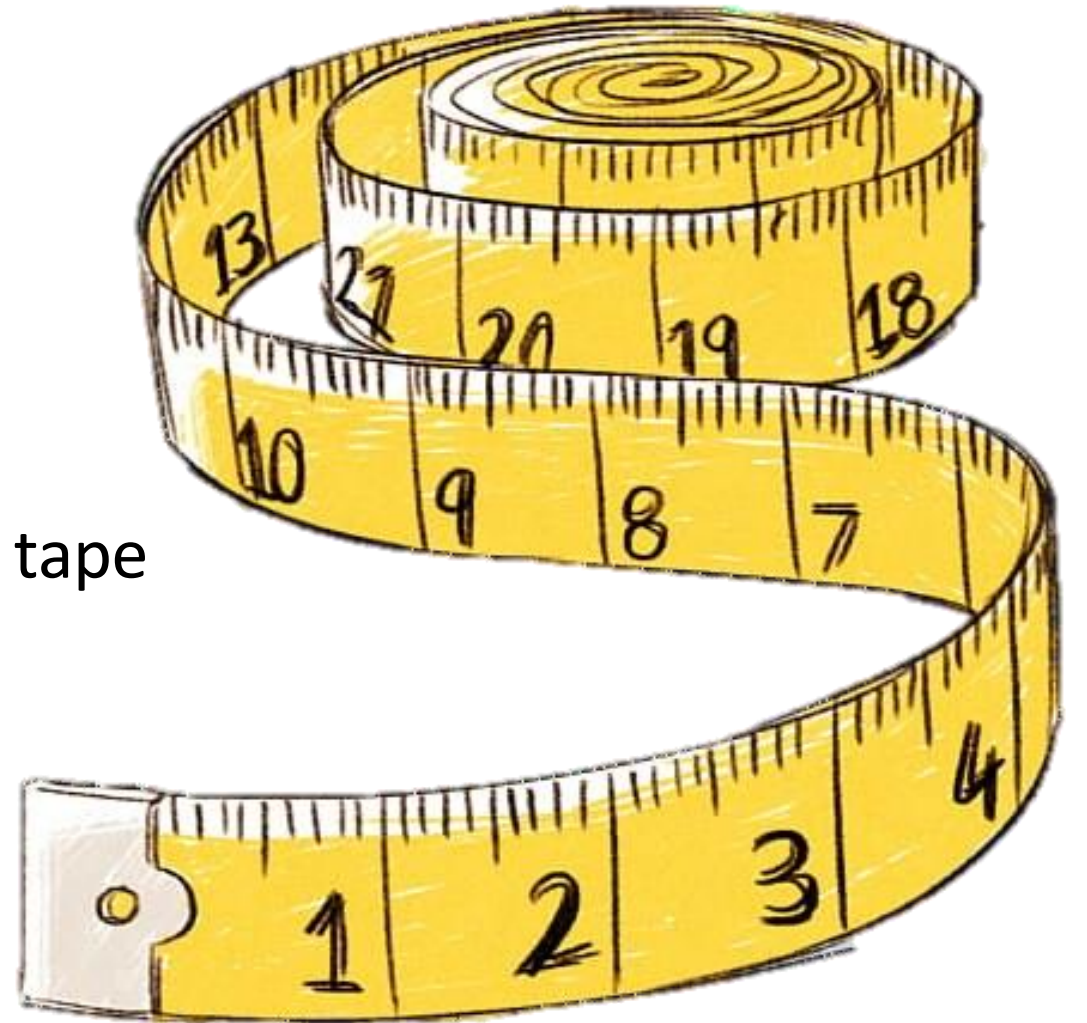
How Memory Works...???

- Each time you want to store an item in memory, you ask the computer for some space, and it gives you an address where you can store your item.
- If you want to store multiple items, there are two basic ways to do so: arrays and lists.



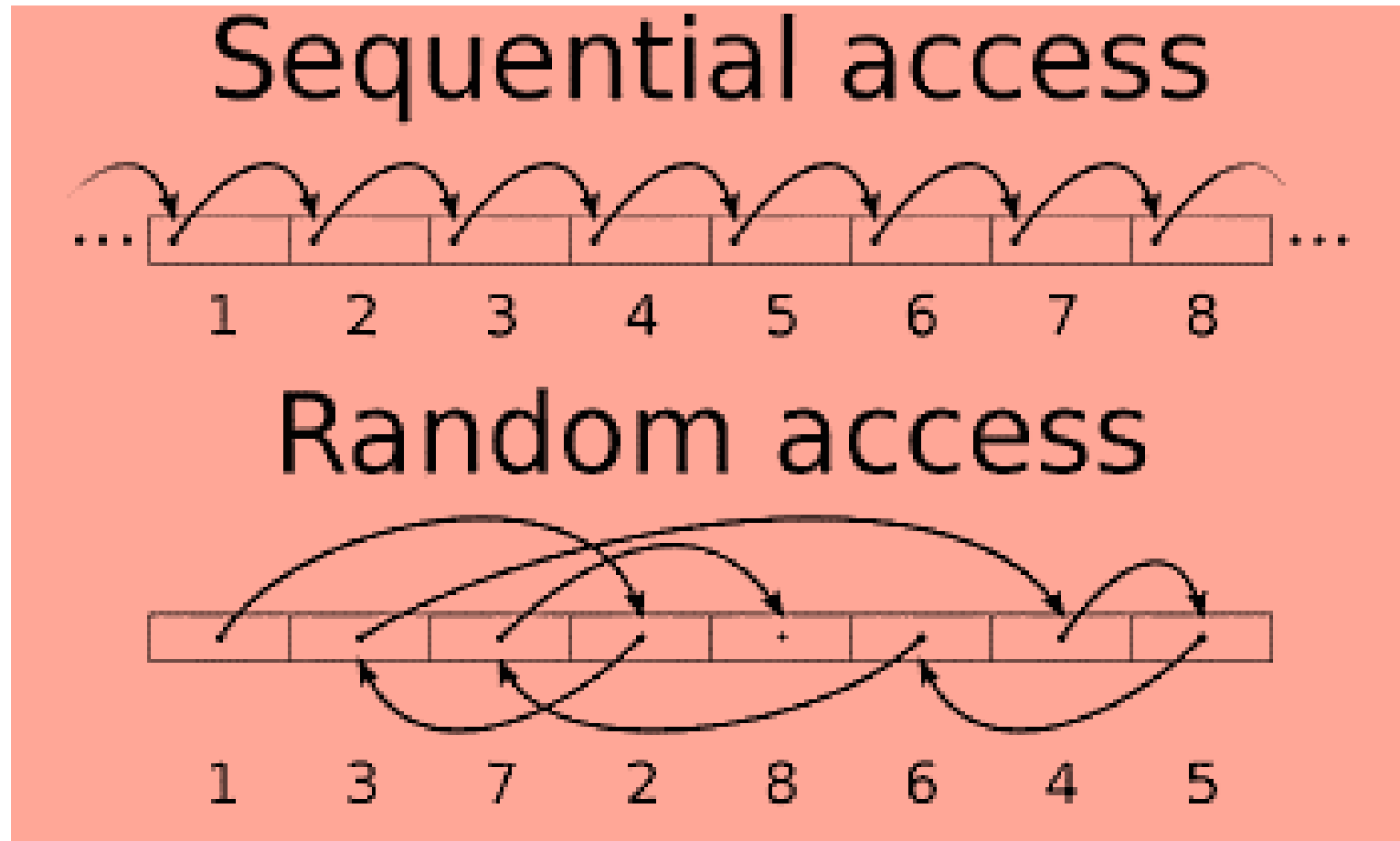
How Memory Works...???

- Now you have understood how, computer memory is organized.
- You can think of memory as a large tape of locations.

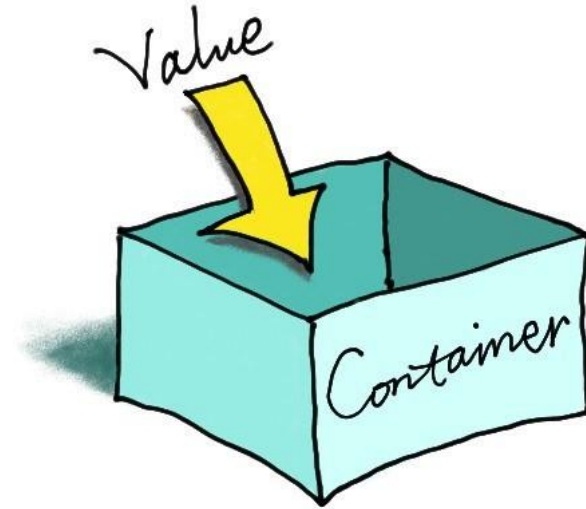


How Memory Works...???

- Any location on this tape can be randomly accessed by its address.



Variable

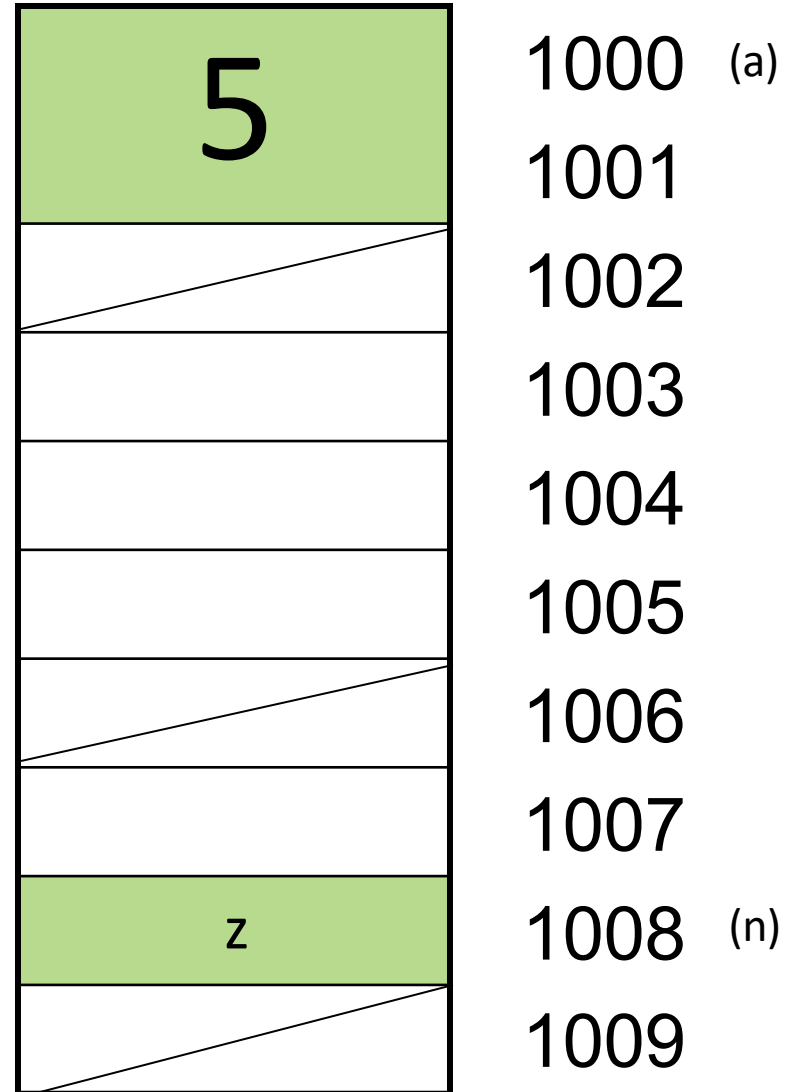


- Placeholder/Structure to store a basic unit of data.
- Example:
 - 1 integer variable stores single integer data.
 - 1 float variable stores single float data.
- Question is:
 - In a computer, where is that value assigned to a variable stored?
 - In memory, possibly Random Access Memory (RAM)
 - But where exactly in memory?
 - On some physical address in memory because,
 - » Memory is divided in physical locations and,
 - » Each location has a particular address associated with it.

Variabl

int a = 5;

char n = 'z';



Array

- Need:
 - Sometimes in our program,
 - There might be a need to store multiple data of similar type.
 - Example:
 - Roll numbers of 10 students.
 - Marks of DSA of 10 students.
 - 2 solutions:
 - Create different variables each having a different name.
 - `int num1, num2, num3` etc.
 - Create a collection of variables referred by a common name.
 - `int num[3]`
 - This looks much better solution.

Array

- Array:
 - Finite, Ordered collection of Homogeneous data elements where,
 - Ordering is maintained by storing all elements in,
 - Continuous / Contiguous locations of memory.
 - Properties:
 - Finite:
 - Contains only a limited (finite) number of elements.
 - Ordered:
 - All elements are stored one by one in continuous / contiguous locations of computer memory.
 - Homogeneous:
 - All the elements of an array are of the same data type.
 - Example:
 - An array of integers to store the roll numbers of all students in a class.
 - An array of strings to store the names of all villagers in a village.

Array

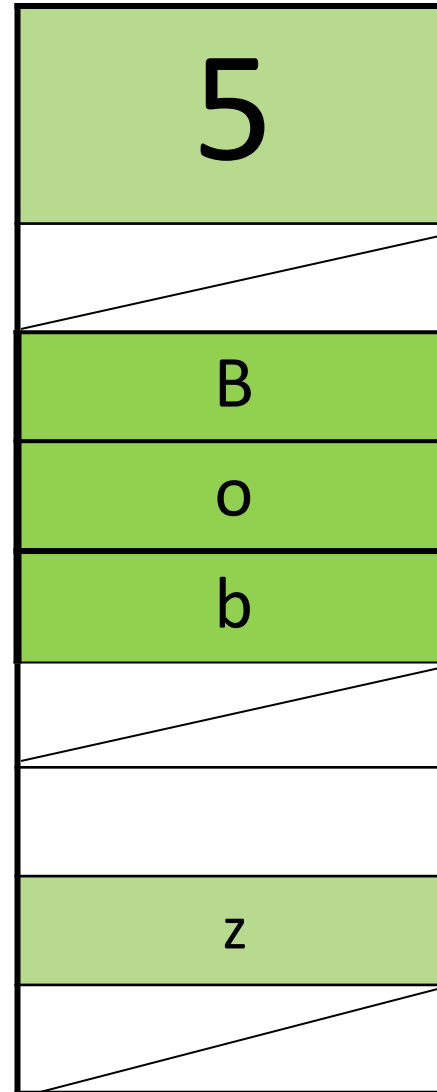
short int a = 5;

char n = 'z';

char name[3] = {'B', 'o', 'b'};

Finite

Same data type
/
Homogeneous



1000 (a)

1001

1002

1003 (name)

1004

1005

1006

1007

1008 (n)

1009

Ordered

Linear Arrays

- A linear array is a list of finite numbers **n** of homogenous data elements . Array has a set of **index** and **values**.

- For each index, there is a value associated with that index.
- Index is used to reference an element in memory location.
- Values are data elements stored in memory.

Linear Arrays

- Implemented by using consecutive memory.
- The number n of elements is called the **length or size** of the array, if not defined we assume index started from **1,2.....,n**. in general the length or the number of data elements of array can be obtained from index set by formula

$$\text{Length} = \text{UB} - \text{LB} + 1$$

- UB is upper bound the largest index, and LB is lower bound the smallest index.

Linear Arrays

-Elements may of an array A denoted by subscript notation

$A_1, A_2, \text{-----} A_n$

- $A[1], A[2] \text{-----} A[n]$ we usually use subscript notation to represents.

Example

- An Automobile company uses an Array AUTO to Record the number of automobiles sold each year from 1932 to 1984.
- Index set begin with 1932 rather than 1.

How many elements are present in AUTO ?

Representation of Linear arrays in memory

- let LA be a linear array in memory of the computers, and memory of computer is simply a sequence of address location and we use notation

$\text{LOC}(\text{LA}[K])$ = address of element $\text{LA}[K]$ of the array LA

- First element of LA, Denoted by **$\text{Base}(\text{LA})$** and called the base address of LA.

Representation of Linear arrays in memory

- computer calculates the address of elements of LA by following formula

$$\text{LOC(LA[K])} = \text{Base (LA)} + w(\text{K} - \text{lower bound})$$

where **w** is the number of words per memory cell for the array LA.

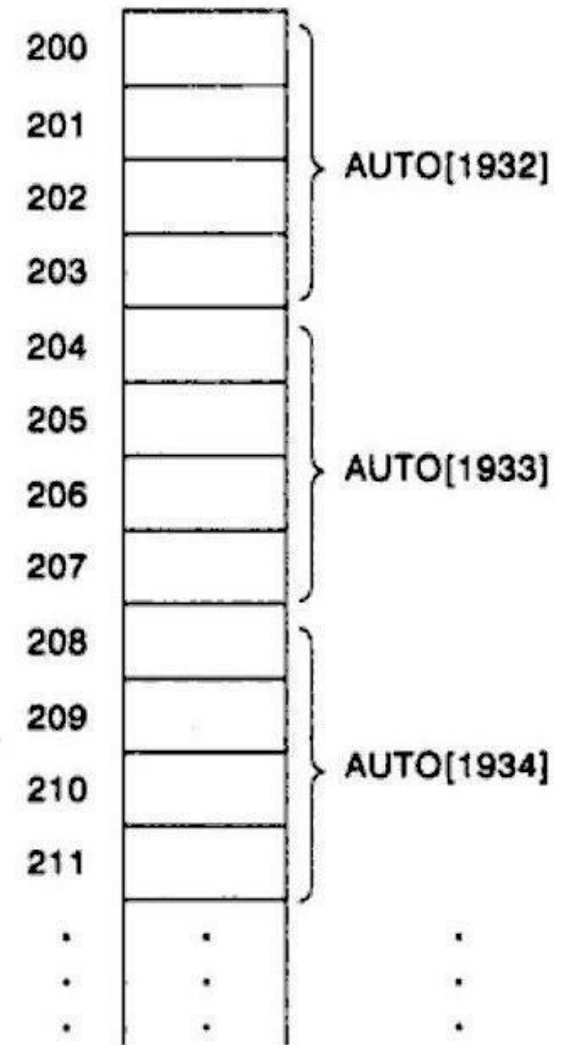
Example 1

AUTO which records the number of automobiles sold each year from 1932 to 1984.

$\text{Base}(\text{Auto}) = 200$

$W = 4$ words per memory cell

**Find the address of element which store
The info about sale in year 1965?**



Example 2

Find the address for LA[6]
Each element of the array occupy
1 byte

200		LA[1]
201		LA[2]
202		LA[3]
203		LA[4]
204		LA[5]
205		LA[6]
206		LA[7]
207		LA[8]

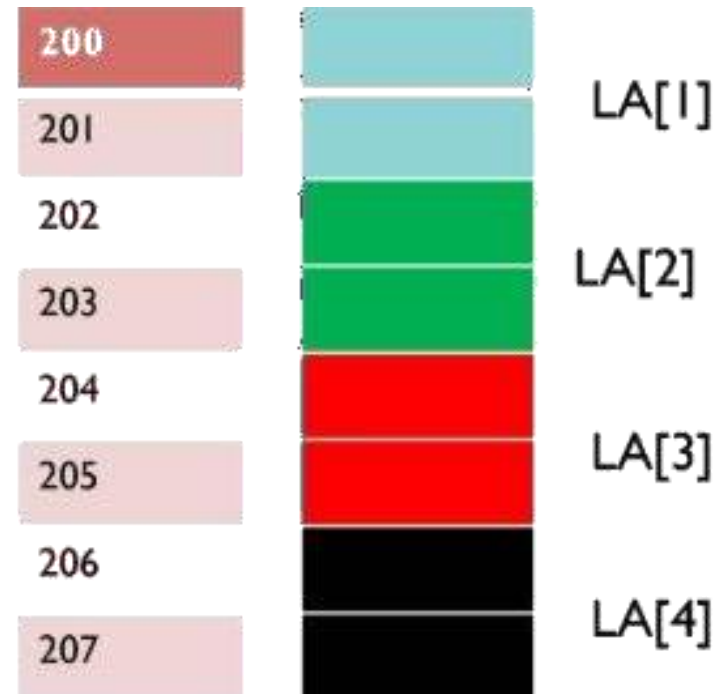
$$\text{LOC}(\text{LA}[K]) = \text{Base}(\text{LA}) + w(K - \text{lower bound})$$

⋮

$$\text{LOC}(\text{LA}[6]) = 200 + 1(6 - 1) = 205$$

Example 3

Find the address for LA[16]
Each element of the array occupy
2 byte



$$\text{LOC}(\text{LA}[K]) = \text{Base}(\text{LA}) + w(K - \text{lower bound})$$

$$\text{LOC}(\text{LA}[16]) = 200 + 2(16 - 1) = 230$$

Basic Operations – Linear Array

Following are the basic operations supported by an array.

Traverse	Print all the array elements one by one.
Insertion	Adds an element at the given index
Deletion	Deletes an element at the given index
Search	Searches an element using the given index or by the value.

Traversing Linear Arrays

Traversing Linear Arrays

- Let B be a collection of data elements stored in memory, and we want to print the content of each elements of B or we want to count the numbers of elements of B. this can be done by Traversing B, that is by **accessing and processing each elements of B exactly once.**

Algorithm - Traversing Linear Arrays

Traversing Linear Array , LA is an array with UB and LB. the algorithm traverses LA

1. Set $K := LB$
 2. repeat step 3 and 4 while $K \leq UB$
 - 3 apply process to $LA[K]$
 4. set $K := K + 1$
- End of step 2
5. Exit

Alternate Algorithm - Traversing Linear Arrays

Traversing Linear Array , LA is an array with UB and LB. the algorithm traverses LA

1. Repeat for $K = LB$ to UB

 Apply process to $LA[K]$

[End of loop]

2. Exit

Traversal

```
void traverse(int arr[], int n) {  
    for(int i = 0; i < n; i++)  
        cout << arr[i] << " ";  
    cout << endl;  
}
```

Exercise

An Automobile company uses an Array AUTO to Record the number of automobiles sold each year from 1932 to 1984.

Write an algorithm to solve the following

- (a) Find the number NUM of years during which more than 300 cars were sold**
- (b) Print each year and the number of cars sold in that year**

Insertion in Linear Arrays

Insertion in Linear Arrays

Inserting an element at the end of a linear array can be easily done provided the memory space allocated for the arrays is large enough to accommodate the additional element. If we need to insert an element in the middle of array. Then on the average half of the elements must be moved downward to new location to accommodate the new elements and keep the order of other elements

Insertion in Linear Arrays

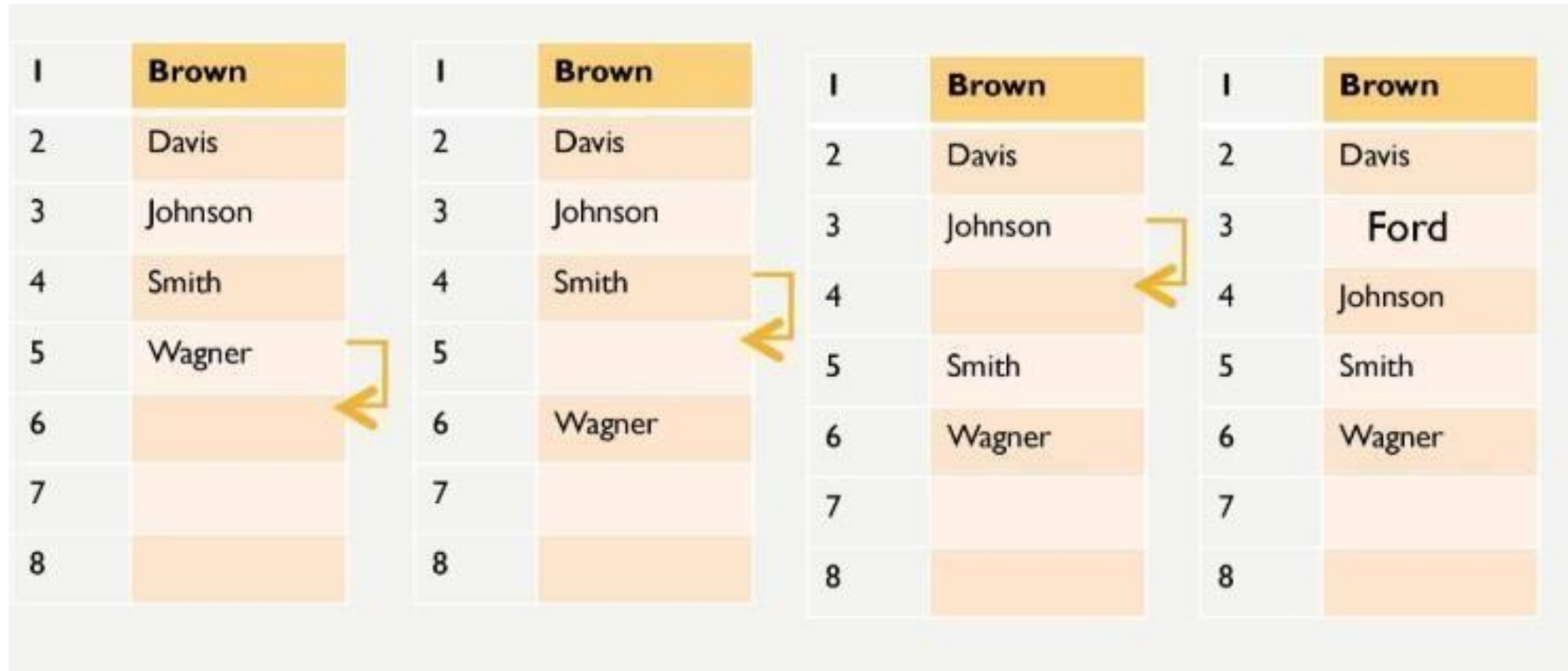
Inserting “Ford” at the end of the Array

1	Brown
2	Davis
3	Johnson
4	Smith
5	Wagner
6	
7	
8	

1	Brown
2	Davis
3	Johnson
4	Smith
5	Wagner
6	Ford
7	
8	

Insertion in Linear Arrays

Inserting “Ford” at the 3rd Element of the Array



Algorithm - Insertion in Linear Arrays

Algorithm of inserting into linear array. INSERT (LA,N, K,
ITEM)

- 1.Set $J:=N$
- 2.Repeat step 3 and 4 while $J \geq K$
- 3.[Move j^{th} element downward] set $LA[J+1] := LA[J]$
- 4.Decreases counter set $J:= J-1$ end of step 2 loop
- 5.Insert element set $LA[K] := \text{ITEM}$
- 6.[Reset N] $N:= N+1$
- 7.EXIT

Insertion

```
int insertAtIndex(int arr[], int n, int x, int pos) {  
    for(int i = n; i > pos; i--)  
        arr[i] = arr[i-1];  
    arr[pos] = x;  
    return n+1;  
}
```

Deletion in Linear Arrays

Deletion in Linear Arrays

Deleting element at end is not difficult but deletion in middle of array would require that each subsequent element be moved one location upward in order to fill up the array.

Deletion in Linear Arrays

Delete “Wagner” at the end of the array.

1	Brown
2	Davis
3	Ford
4	Johnson
5	Smith
6	Taylor
7	Wagner
8	

1	Brown
2	Davis
3	Ford
4	Johnson
5	Smith
6	Taylor
7	
8	

Deletion in Linear Arrays

Deletion of "Davis" from the Array

1	Brown	1	Brown	1	Brown	1	Brown
2	Davis	2		2	Ford	2	Ford
3	Ford	3	Ford	3		3	Johnson
4	Johnson	4	Johnson	4	Johnson	4	
5	Smith	5	Smith	5	Smith	5	Smith
6	Taylor	6	Taylor	6	Taylor	6	Taylor
7	Wagner	7	Wagner	7	Wagner	7	Wagner
8		8		8		8	

Deletion in Linear Arrays

DELETE (LA, N, K, ITEM)

Here LA is a linear array with N elements and K is a positive integer such that $K \leq N$. This algorithm deletes the Kth element from LA.

1. Set $ITEM = LA[K]$
2. Repeat from $J = K$ to $N - 1$
 [Move $J + 1^{st}$ element upward] Set $LA[J] = LA[J + 1]$
 [End of loop]
3. [Reset the number N of elements in A] Set $N = N - 1$
4. Exit

Deletion

```
int deleteAtIndex(int arr[], int n, int pos) {  
    for(int i = pos; i < n-1; i++)  
        arr[i] = arr[i+1];  
    return n-1;  
}
```

Search Operation in Linear Arrays

Search Operation

You can perform a search for an array element based on its value or its index.

Consider **LA** is a linear array with

N elements and

K is a positive integer such that

$K \leq N$

1. Start
2. Set $J = 0$
3. Repeat steps 4 and 5 while $J < N$
4. IF $LA[J]$ is equal ITEM THEN GOTO STEP 6
5. Set $J = J + 1$
6. PRINT J, ITEM
7. Stop

ADVANTAGES AND DISADVANTAGES

- **Advantage**

- Fast, random access of elements.
- Very memory efficient, very little memory is required other than that needed to store the contents.

- **Disadvantage**

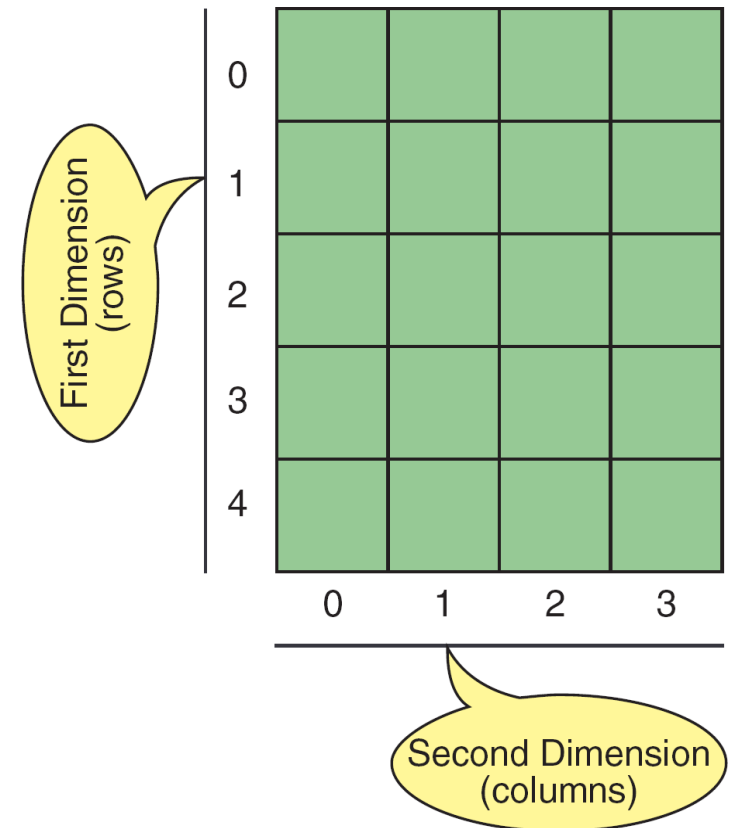
- Slow deletion and insertion of elements.
- **Size must be known when the array is created and is fixed (static).**

APPLICATION OF ARRAYS

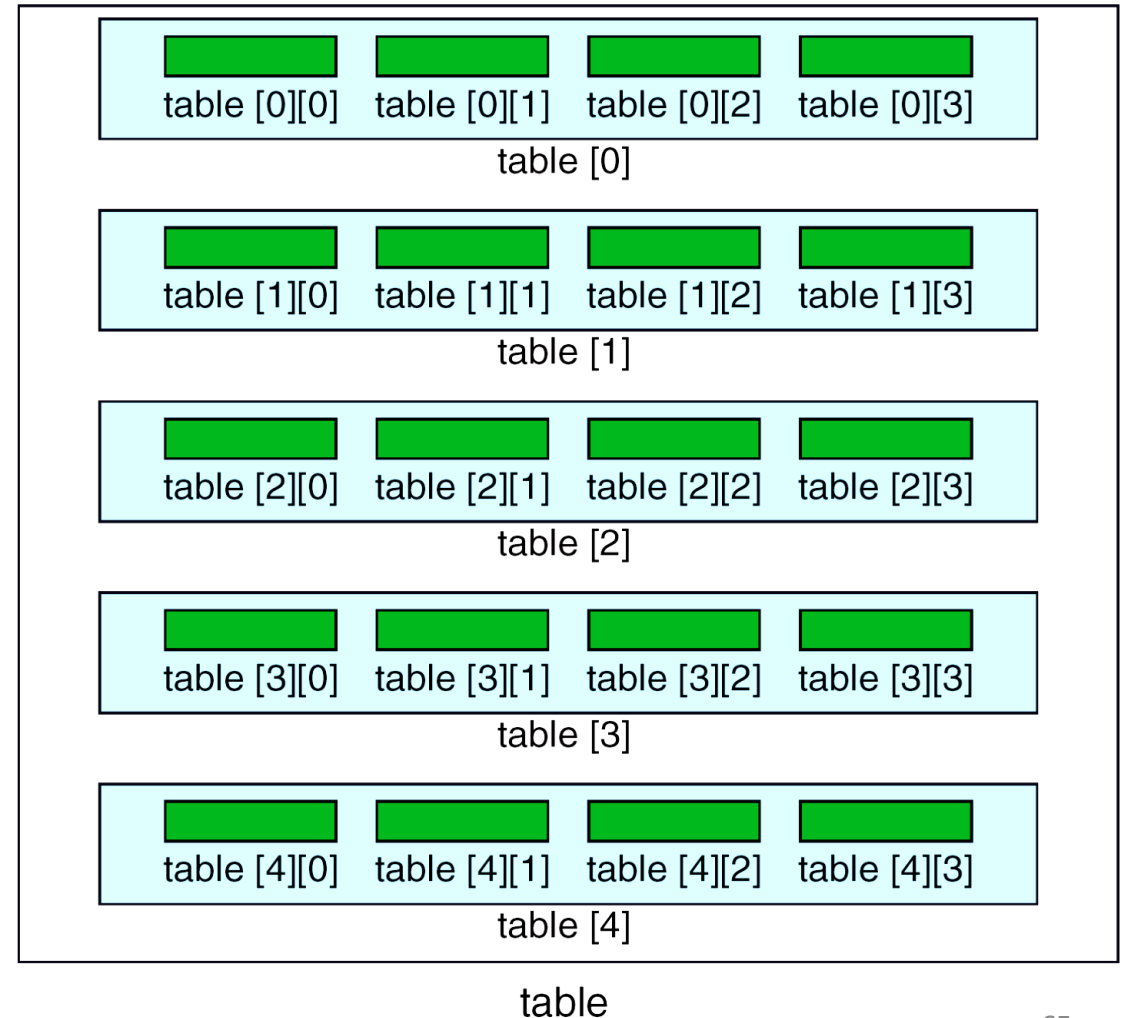
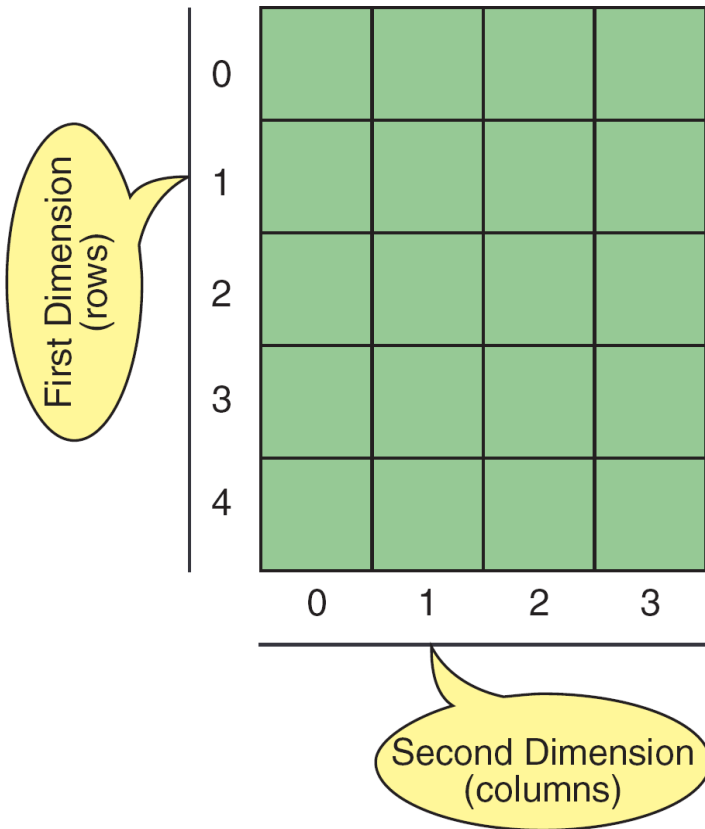
- Arrays are used to implement mathematical vectors and matrices, as well as other kinds of rectangular tables.
- Many databases, small and large, consist of one-dimensional arrays whose elements are records.
- Arrays are used to implement other data structures, such as:
 - Lists
 - Heaps
 - Hash tables
 - Queues
 - Stacks
 - Etc.

Two-Dimensional Array

- *The arrays we have discussed so far are known as one-dimensional arrays because the data are organized linearly in only one direction.*
- *Many applications require that data be stored in more than one dimension.*
- *One common example is a table, which is an array that consists of rows and columns.*



Two-Dimensional Array (*An Array of Arrays*)



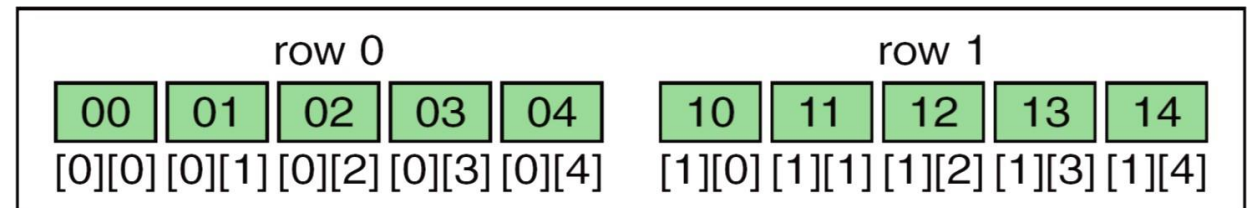
Two-Dimensional Array



- This representation is called the **array-of-arrays** representation.
- Requires i contiguous memory of size k .
- k is the number of columns in each row.
- Suppose a 2D Array with 2 rows and 5 columns.
 - *Its logical and Physical view are shown below.*

00	01	02	03	04
10	11	12	13	14

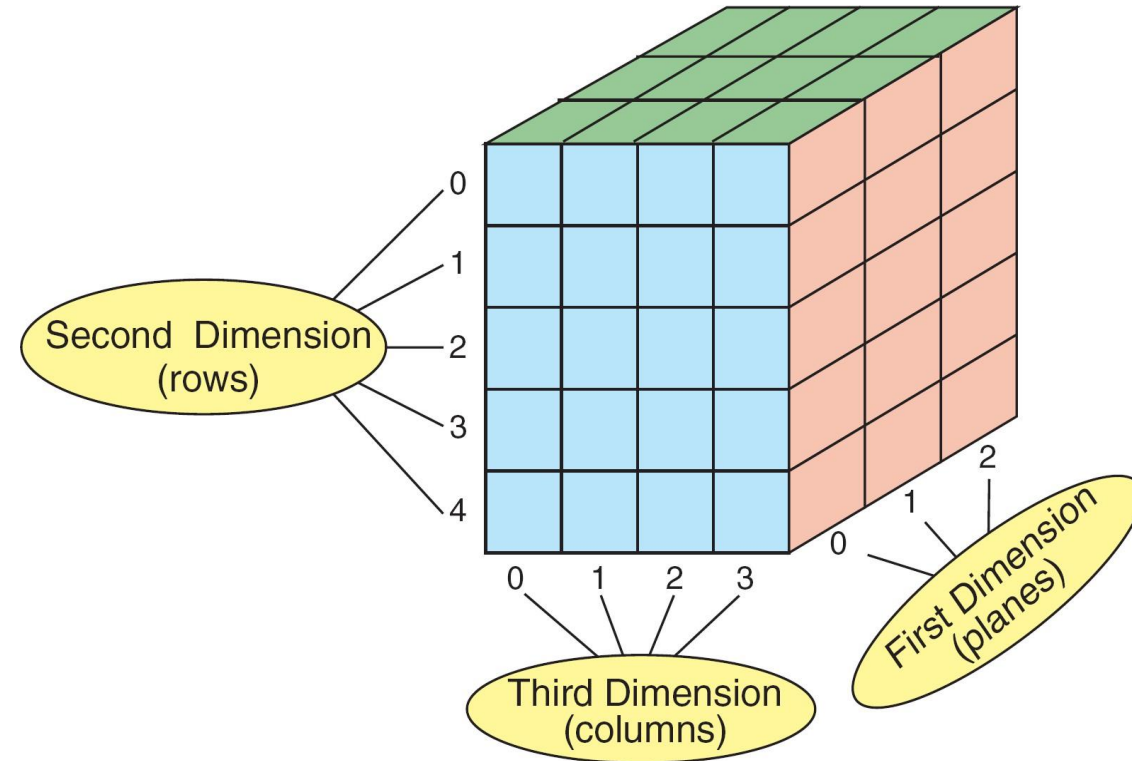
User's View



Memory View

Multi-Dimensional Array

- *Multidimensional arrays can have three, four, or more dimensions.*
- *The first dimension is called a plane, which consists of rows and columns.*



Multi-Dimensional Array

- *The C language considers the three-dimensional array to be an array of two-dimensional arrays.*
- *The same concept can be extended to 4D arrays and above.*

